COLEGIO UNIVERSITARIO DE CARTAGO

DEPARTAMENTO DE TECNOLOGÍAS DE LA INFORMACIÓN

CURSO: PROGRAMACIÓN V (COD. P5)

TAREA CORTA 2
INVESTIGACIÓN ARQUITECTURA DE API GATEWAYS Y MICROSERVICIOS

Diseño, Patrones, Resiliencia y Evaluación Comparativa

Integrantes del Equipo de Trabajo:

- Héctor Román Arrieta
- Ramsés Aguilar Monge
- Alejandro Morales
- Fabián Espinoza Méndez
- José Rodolfo Morales Calderón

Docente: Prof. Ing. Gabriel González Solano

Fecha de Entrega: 15 de Octubre de 2026

Periodo Lectivo: III Cuatrimestre 2026

Introducción

En la evolución del desarrollo de software empresarial, la transición desde arquitecturas monolíticas centralizadas hacia sistemas distribuidos basados en microservicios ha permitido a las organizaciones alcanzar altos niveles de escalabilidad horizontal, despliegues continuos independientes y desacoplamiento de dominios comerciales (Newman, 2021). Sin embargo, esta descomposición introduce una complejidad de red inherente: decenas o cientos de servicios autónomos, cada uno con sus propias direcciones IP, puertos, protocolos y esquemas de datos.

Si los clientes externos (aplicaciones web SPA, dispositivos móviles nativos, sistemas de terceros) intentaran comunicarse directamente con cada microservicio individual, el sistema colapsaría bajo problemas severos de seguridad, acoplamiento extremo, sobrecarga de solicitudes de red y dificultades insalvables de gobernanza. Para abordar esta problemática estructural surge el patrón de **API Gateway**.

1. Definición de API Gateway y su Propósito

Un **API Gateway** es un componente de software arquitectónico que se sitúa como intermediario o punto de entrada único (*Single Entry Point*) entre los clientes externos y un conjunto de microservicios internos que componen una aplicación distribuida (Richardson, 2018).

El propósito primordial del API Gateway es abstraer la topología y complejidad interna del ecosistema de microservicios, proporcionando a los consumidores una interfaz consolidada, uniforme, segura y optimizada. En términos conceptuales, actúa como el "director de orquesta" y la "frontera perimetral" de la plataforma.

2. Problemas que Resuelve y Diferencias con un Reverse Proxy Tradicional

Uno de los equívocos más frecuentes en ingeniería de software consiste en equiparar un API Gateway con un Reverse Proxy tradicional (como una instancia estándar de NGINX o HAProxy). Aunque ambos componentes interceptan y redirigen tráfico HTTP, operan en diferentes capas de abstracción y resuelven problemas con alcances totalmente distintos.

> **Diferencia Fundamental:** Un reverse proxy tradicional se enfoca en la optimización del transporte de paquetes de red (capas 4 y 7 básica), entrega de archivos estáticos y balanceo de carga; un **API Gateway** opera en la capa de aplicación superior, orquestando lógica de composición, validación criptográfica de identidades (JWT/OAuth2), políticas de negocio y transformación de estructuras JSON.

Criterio	Reverse Proxy Tradicional (NGINX clásico)	API Gateway Moderno (Kong, Ocelot, YARP)
Capa Principal de Operación	Capas 4 (Transporte) y 7 básica (HTTP estándar).	Capa 7 avanzada (Capa de Aplicación y Dominio Ligero).
Enrutamiento	Estático, mediante archivos planos de configuración.	Dinámico, reactivo a catálogos de Service Discovery (Consul, K8s).
Composición y Agregación	No disponible o compleja mediante scripts C/Lua.	Nativa: combina datos de múltiples microservicios en una sola respuesta.
Seguridad de Aplicación	Inspección TLS y listas de control de acceso IP.	Validación de JWT descentralizado, integración OIDC y claims.
Transformación	Reescritura simple de cabeceras.	Reescritura de cuerpos JSON, mutación de rutas y conversión de protocolos.

3. Funciones Comunes del API Gateway

A continuación se detallan diez de las capacidades más determinantes que provee un API Gateway:

1. **Enrutamiento Dinámico y Proxy Inverso:** Inspecciona el verbo HTTP y la URI entrante para redirigir la petición a la dirección interna y puerto del microservicio correspondiente.
2. **Autenticación y Autorización Centralizada:** Valida firmas criptográficas de tokens JWT y rechaza peticiones no autenticadas en el perímetro antes de saturar la red interna.
3. **Limitación de Tasa (Rate Limiting y Throttling):** Aplica algoritmos formales (Token Bucket / Leaky Bucket) para restringir el número de solicitudes por segundo por cliente o IP, previniendo ataques de denegación de servicio.
4. **Almacenamiento en Caché (Caching):** Almacena temporalmente respuestas HTTP de lecturas frecuentes para servirlas de inmediato sin tocar los microservicios aguas abajo.

5. **Terminación SSL/TLS:** Gestiona certificados y negociación HTTPS perimetral, permitiendo comunicación interna ligera y eficiente.
6. **Balanceo de Carga (Load Balancing):** Reparte la carga entre instancias saludables de microservicios usando algoritmos como Round-Robin o Least Connections.
7. **Transformación de Peticiones y Respuestas:** Inyecta cabeceras de trazabilidad (`X-Request-Id`), oculta cabeceras internas confidenciales y reescribe URLs para retrocompatibilidad.
8. **Agregación de Datos (BFF / API Composition):** Realiza llamadas paralelas a múltiples servicios internos y fusiona los resultados en un solo JSON optimizado para el cliente.
9. **Observabilidad Centralizada:** Genera identificadores de correlación distribuidos (Distributed Tracing), registros de auditoría y métricas de latencia de red.
10. **Resiliencia y Disyuntor (Circuit Breaking):** Detecta caídas en microservicios downstream e interrumpe llamadas recursivas enviando respuestas de fallback controladas.

4. Ventajas y Desventajas

Ventajas: Desacoplamiento total entre frontend y topología interna de servidores; centralización de seguridad; disminución drástica de peticiones en red móvil (chatty I/O); y gobernanza simplificada.

Desventajas y Riesgos:

- **Punto Único de Falla (SPOF):** Una caída del Gateway inhabilita el acceso a toda la plataforma si no cuenta con alta disponibilidad.
- **Latencia Adicional:** El procesamiento perimetral y el salto de red adicional añaden milisegundos a la transacción.
- **Riesgo de Acoplamiento (Antipatrón Monolito Oculto):** Si los desarrolladores introducen lógica pesada de reglas de negocio en el Gateway, este degenera en un monolito difícil de mantener.

5. Patrones Arquitectónicos Relacionados

- **Backend for Frontend (BFF):** Creación de gateways especializados por tipo de cliente (móvil, web SPA, terceros B2B), permitiendo formatos y contratos a la medida de cada pantalla.
- **Patrón Higo Estrangulador (Strangler Fig):** Estrategia para migrar monolitos hacia microservicios enrutando gradualmente funcionalidades al nuevo stack sin cortes de servicio.

- **Composición de API (API Composition):** Mecanismo para consolidar datos cuando se aplica el principio de base de datos aislada por servicio (Database-per-Service).
- **Descubrimiento de Servicios (Service Discovery):** Registro dinámico de instancias efímeras en entornos de contenedores elásticos (Kubernetes o Consul).

6. Comparativa de Herramientas de API Gateway

Herramienta	Lenguaje / Stack	Tipo	Curva de Aprendizaje	Funciones Clave	Caso Ideal de Uso
Ocelot	C# / .NET (ASP.NET Core)	Open-source	Baja para desarrolladores .NET	Configuración declarativa JSON, agregación nativa de endpoints, caché, autenticación JWT integrada y balanceo de carga.	Sistemas empresariales en .NET que buscan una solución ligera e integrada por middleware.
Kong Gateway	Lua / C / OpenResty (NGINX)	Open-source / Enterprise	Media	Rendimiento submilisegundo, catálogo masivo de plugins oficiales, arquitectura sin estado (DB-less mode) y mTLS.	Ecosistemas de alta concurrencia heterogéneos con cientos de miles de peticiones por segundo.
YARP	C# / .NET 8 y 9	Open-source (Microsoft)	Baja para desarrolladores .NET	Rendimiento de vanguardia, soporte nativo de HTTP/3, extensibilidad programática completa en C# y balanceo activo.	Sistemas modernos en .NET que requieren alto rendimiento y lógica personalizada en C#.
NGINX	C	Open-source / Commercial	Media a Alta	Rendimiento legendario, consumo mínimo de RAM, terminación SSL muy veloz y balanceo en capas 4 y 7.	Punto de entrada perimetral de infraestructura y balanceador

Herramienta	Lenguaje / Stack	Tipo	Curva de Aprendizaje	Funciones Clave	Caso Ideal de Uso
					general de clústeres.
Traefik	Go	Open-source / Commercial	Baja	Autodescubrimiento nativo con Docker y Kubernetes, certificados Let's Encrypt automáticos y dashboard web en vivo.	Entornos de contenedores y microservicios nativos en la nube sin configuración manual de rutas.
Spring Cloud Gateway	Java / Kotlin (Spring Boot)	Open-source	Media a Alta	Enrutamiento no bloqueante basado en Project Reactor, integración con Eureka, Spring Security y Resilience4j.	Corporaciones cuyo stack tecnológico principal reside en Java y el ecosistema Spring.

7. Conclusiones y Recomendaciones

Cuándo usar un Gateway: Cuando el sistema cuenta con más de 3 microservicios, existen diversos canales de clientes (móvil, web, API pública), se requiere una frontera de seguridad homogénea y auditoría perimetral de tráfico.

Cuándo evitarlo: En sistemas monolíticos consolidados, arquitecturas con 1 o 2 microservicios triviales donde la sobrecarga de mantenimiento supera los beneficios, o en sistemas con tolerancia cero a la latencia de red (como procesamiento de transacciones financieras de alta frecuencia - HFT).

8. Matriz de Responsabilidades y División de Trabajo del Equipo

De acuerdo con los lineamientos de la rúbrica del curso, a continuación se detalla la asignación de responsabilidades y el porcentaje de participación individual en el diseño,

desarrollo, pruebas y documentación:

Miembro del Equipo	Rol Principal	Módulo Asignado	% Part.	Entregables Concretos
Ramsés Aguilar Monge	Líder Técnico & Arquitectura	API Gateway Centralizado y Enrutamiento	20%	Diseño del Gateway en puerto 8080, middleware de enrutamiento perimetral, transformación de cabeceras (`X-Gateway`, `X-Request-Id`) y logging de observabilidad.
José Rodolfo Morales Calderón	Backend & Integración	Política de Agregación y Pruebas Postman	20%	Implementación del endpoint de agregación BFF (`/api/resumen-orden/:id`), agregación de dashboard general (`/api/dashboard`) y suite completa en Postman v2.1.
Fabián Espinoza Méndez	Backend Developer	Microservicio de Órdenes e Inter-Servicio	20%	API REST en puerto 5003, modelo NoSQL en MongoDB (`cuc_ordenes_db`), comunicación inter-servicio con Productos y Clientes para validación de stock y precios.
Héctor Román Arrieta	Backend Developer	Microservicio de Productos / Catálogo	20%	API REST en puerto 5001, modelo NoSQL en MongoDB (`cuc_productos_db`), endpoints CRUD, lógica de inventario y endpoint de descuento de stock.
Alejandro Morales	Backend & Documentación	Microservicio de Clientes e Investigación Teórica	20%	API REST en puerto 5002, modelo NoSQL en MongoDB (`cuc_clientes_db`), script de seed de datos y redacción formal del informe teórico en formato APA.
TOTAL PARTICIPACIÓN DEL EQUIPO:			**100%**	

Referencias Bibliográficas (APA 7ª Edición)

Fowler, M. (2014). *Microservices: a definition of this new architectural term*. MartinFowler.com. https://martinfowler.com/articles/microservices.html

Fowler, M. (2020). *StranglerFigApplication*. MartinFowler.com. https://martinfowler.com/bliki/StranglerFigApplication.html

Kong Inc. (2023). *What is an API Gateway? Architecture, patterns, and best practices*. KongHQ Documentation. https://konghq.com/learning-center/api-gateway/what-is-an-api-gateway

Microsoft Corporation. (2024). *Diseño de arquitectura de microservicios: Implementación de API Gateways con YARP y Ocelot*. Microsoft Learn Architecture Center. https://learn.microsoft.com/es-es/azure/architecture/microservices/design/gateway

NGINX Inc. (2022). *Deploying NGINX as an API Gateway: Comprehensive architectural guide*. F5 Networks Technical Reports. https://www.nginx.com/blog/deploying-nginx-api-gateway/

Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.

Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.

Richardson, C. (2022). *Pattern: API Gateway / Backends for Frontends*. Microservices.io. https://microservices.io/patterns/apigateway.html

Traefik Labs. (2024). *Traefik The Cloud Native Application Proxy: Architecture and core concepts*. Traefik Docs. https://doc.traefik.io/traefik/